

Linux & Operating Systems Programming Roadmap

This roadmap is intended for learning how Linux and operating systems work internally while improving your C programming skills. Instead of immediately building a full operating system, the goal is to understand how programs interact with Linux: - processes - memory - files - system calls - permissions - signals Debian is an excellent environment for learning these concepts because Linux exposes many system internals openly.

Stage 1 — Linux System Interaction

Focus on:

- Processes
- Memory
- Files
- System calls

Goal:

Understand how normal programs communicate with the Linux kernel.

Project 1 — Linux System Monitor

Build a small terminal system monitor.

Features:

- CPU information
- RAM usage
- System uptime
- Running processes

Useful Linux files:

/proc/cpuinfo
/proc/meminfo
/proc/uptime

Functions to explore:

fopen()
fgets()
fscanf()

Useful commands:

top
htop
cat /proc/cpuinfo
cat /proc/meminfo

Project 2 — Mini Shell

Create your own Linux shell.

Features:

- Accept commands
- Execute Linux programs
- Support cd
- Support exit

Important functions:

fork()
execvp()
wait()

Critical concept:

A shell creates child processes using fork().

Project 3 — File Permissions Explorer

Learn Linux file permissions and metadata.

Features:

- Show permissions
- Show owner/group
- Show file size
- Show modification time

Functions:

stat()

chmod()

Important permission format:

rw-r--r--

Project 4 — Process Manager

Learn process control and signals.

Features:

- List running processes
- Kill process by PID

Functions:

kill()

signal()

Signals:

SIGTERM

SIGKILL

SIGINT

Project 5 — Memory Allocator

Create a simplified version of malloc/free.

Concepts:

- Heap memory
- Allocation tracking
- Fragmentation
- Free lists

This project explains how memory allocation works internally.

Core Linux Concepts

1. Everything is a file

Devices, terminals, processes, and sockets often behave like files.

2. User space vs kernel space

Programs run in user space while the kernel controls hardware and system resources.

3. System calls

Functions such as:

read()

write()

fork()

open()

eventually communicate with the Linux kernel.

Recommended Learning Resources

Website	Purpose
https://man7.org/linux/man-pages/	Excellent Linux man pages
https://linuxjourney.com/	Beginner-friendly Linux learning
https://wiki.archlinux.org/	Deep Linux explanations
https://www.gnu.org/software/libc/manual/	GNU C library documentation
https://beej.us/guide/	Systems/network programming guides
https://kernelnewbies.org/	Linux kernel beginner resources
https://explainshell.com/	Explains Linux terminal commands

Terminal Exploration

Useful Commands To Explore

```
man fork
man execve
man proc
man signal
strace ls
ps aux
ls /proc
```

One of the best ways to learn Linux internals:

1. Use the command yourself
2. Observe the output
3. Recreate part of it in C

Final Notes

Recommended Progression:

1. C basics
2. Pointers
3. Memory management
4. File I/O
5. Processes
6. System calls
7. Threads
8. Networking
9. Kernel concepts
10. Low-level OS development

Avoid jumping directly into building a full operating system.
Understanding Linux internals first provides a much stronger foundation.